

1. Architettura generale

L'applicazione Leaveat adotta un'architettura di tipo client-server basata su un modello a tre livelli (presentation layer, application layer, data layer).

1.1 Struttura logica

L'architettura si compone dei seguenti livelli principali:

- **Frontend (Presentation Layer)**
Sviluppato con HTML5, CSS3, Bootstrap e JavaScript.
Gestisce l'interazione con l'utente finale (Customer o Restaurateur).
Comunica con il backend tramite chiamate API REST (HTTP/HTTPS).
Include pagine dedicate a:
 - Login e registrazione utenti
 - Visualizzazione di ristoranti e menù
 - Dashboard cliente (ordini, cronologia, profilo)
 - Dashboard ristoratore (gestione ristoranti, menù, statistiche)
- **Backend (Application Layer)**
Realizzato in Node.js con il framework Express.js.
Espone un insieme di API RESTful per la gestione delle risorse principali (User, Restaurant, Menu, Dish, Order).
Contiene la logica applicativa per:
 - autenticazione e autorizzazione (JWT)
 - validazione e gestione errori
 - aggiornamento degli stati degli ordini
 - calcolo delle statistiche per ristorante
 - importazione iniziale del dataset meals.json
- **Database (Data Layer)**
Basato su MongoDB.
Gestisce i dati strutturati in collezioni corrispondenti ai modelli Mongoose (User, Customer, Restaurateur, Restaurant, Menu, Dish, Order).
Ogni documento rappresenta un'entità logica del dominio con relazioni 1:1 o 1:N implementate tramite ObjectId.
Durante l'avvio, viene eseguita una procedura di setup automatico che importa i dati iniziali dei piatti da meals.json.

1.2 Flusso di interazione

Il client (browser) invia una richiesta HTTP verso un endpoint REST del backend.

Il server Express elabora la richiesta, applica controlli di autenticazione/autorizzazione e interagisce con MongoDB.

I dati vengono restituiti in formato JSON al frontend.

L'interfaccia aggiorna dinamicamente i contenuti (ad es. lista dei ristoranti, stato ordini, ecc.).

2. Modelli dati

```
const UserSchema = new Schema({
  _id: {
    type: Schema.Types.ObjectId,
    auto: true
  },

  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true //rimuove gli spazi ad inizio e fine
  },
  passwordHash: {
    type: String,
    required: true
  },
  firstName: {
    type: String,
    required: true
  },
  lastName: {
    type: String,
    required: true
  },
  phoneNumber: {
    type: String,
    required: true,
    unique: true
  },
  role: {
    type: String,
    enum: ["CUSTOMER", "RESTAURATEUR"],
    required: true
  }
});
```

```

const CustomerSchema = new Schema({
  //ID generato automaticamente
  _id: {
    type: Schema.Types.ObjectId,
    auto: true
  },
  //User collegato (1:1)
  userId: {
    type: Schema.Types.ObjectId,
    ref: "User",
    required: true,
    unique: true
  },
  //Personalizzazione account
  preferences: {
    favoriteCategory: {
      type: String,
      enum: [
        "DESSERT",
        "CHICKEN",
        "BEEF",
        "VEGETARIAN",
        "STARTER",
        "MISCELLANEOUS",
        "SIDE",
        "SEAFOOD",
        "BREAKFAST",
        "PORK",
        "PASTA",
        "GOAT"
      ]
    },
    favoriteRestaurantId: {
      type: Schema.Types.ObjectId,
      ref: "Restaurant"
    }
  },
  //Metodi di pagamento
  paymentMethod: {
    type: String,
    enum: ["CASH", "PREPAID_CARD", "CREDIT_CARD"],
    default: "CASH"
  }
});

```

```
const RestaurateurSchema = new Schema({
  _id:{
    type: Schema.Types.ObjectId,
    auto: true
  },
  //Collegamento user (1:1)
  userId:{
    type: Schema.Types.ObjectId,
    ref: "User",
    required: true,
    unique: true
  },
  //Collegamento ristoranti (1:N)
  restaurantId:[{
    type: Schema.Types.ObjectId,
    ref: "Restaurant",
    required: true
  }],

  VATNumber:{
    type: String,
    required: true,
    unique: true,
    trim: true
  }
});
```

```
const RestaurantSchema = new Schema({
  _id:{
    type: Schema.Types.ObjectId,
    auto: true
  },
  name:{
    type: String,
    required: true,
    trim: true
  },
  phoneNumber:{
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  address:{
    type: String,
    required: true,
    trim: true
  },
  //Collegamento menù (1:1)
  menuId:{
    type: Schema.Types.ObjectId,
    ref: "Menu",
    unique: true
  },
  //Collegamento ordini (0:n)
  orderId:[{
    type: Schema.Types.ObjectId,
    ref: "Order",
  }],
});
```

```
const MenuSchema = new Schema({
  _id:{
    type: Schema.Types.ObjectId,
    auto: true
  },
  //Collegamento ristorante (1:n)
  restaurantId:[{
    type: Schema.Types.ObjectId,
    ref: "Restaurant",
    required: true
  }],
  //Collegamento ristoratore (1:1)
  restaurateurId:{
    type: Schema.Types.ObjectId,
    ref: "Restaurateur",
    required: true,
    unique: true
  },
  //Collegamento piatti (1:n)
  dishId:[{
    type: Schema.Types.ObjectId,
    ref: "Dish",
  }],
});
```

```
const DishSchema = new Schema({
  _id: {
    type: Schema.Types.ObjectId,
    auto: true
  },
  // Per piatti importati da Meal.json
  externalId: {
    type: String, // idMeal
    unique: true,
    index: true
  },
  name: {
    type: String, // strMeal
    required: true,
    trim: true
  },
  category: {
    type: String, // strCategory
    required: true
  },
  area: {
    type: String, // strArea (country of origin)
    trim: true
  },
  instructions: {
    type: String, // strInstructions
    trim: true
  },
  imageUrl: {
    type: String, // strMealThumb
    trim: true
  },
  tags: {
    type: [String], // parsed from strTags (split by comma)
    default: []
  },
  ingredients: {
    type: [String], // combined from strIngredient1...strIngredient20
    required: true
  },
  measures: {
    type: [String], // combined from strMeasure1...strMeasure20
    default: []
  }
});
```

```

    },
    price: {
      type: Number,
      required: true,
      min: 0
    },
    restaurantId: [{
      type: Schema.Types.ObjectId,
      ref: "Restaurant"
    }]
  });

```

```

const OrderSchema = new Schema({
  _id:{
    type: Schema.Types.ObjectId,
    auto: true
  },
  //Collegamento cliente (1:1)
  customerId:{
    type: Schema.Types.ObjectId,
    ref: "CustomerSchema",
    required: true,
    unique: true
  },
  //Collegamento ristorante (1:1)
  restaurantId:{
    type: Schema.Types.ObjectId,
    ref: "RestaurantSchema",
    required: true,
    unique: true
  },
  //Collegamento piatti (1:n)
  mealId:{
    type: Schema.Types.ObjectId,
    ref: "DishSchema",
  },
  price:{
    type: Number,
    required: true,
    unique: true
  }
});

```


3. API REST

L'interfaccia REST rappresenta il punto di contatto tra frontend e backend e consente la comunicazione tramite HTTP/HTTPS in formato JSON.

Ogni risorsa principale del dominio (utente, ristorante, menù, piatto, ordine) è esposta tramite endpoint specifici che rispettano le convenzioni REST.

3.1 Struttura della risposta

3.1.1 Success

```
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully"
}
```

3.1.2 Error

```
{
  "success": false,
  "error": "ValidationError",
  "message": "Email already registered"
}
```

3.2 Gestione dei percorsi

Tutti gli endpoint condividono un prefisso comune: */api/v/*

3.3 Autenticazione e ruoli

Alcune rotte sono accessibili solo a seguito di una autenticazione effettuata tramite JWT (JSON Web Token).

Sono accessibili senza alcuna autenticazione gli endpoint:

- */restaurants*
- */dishes*

3.4 Tipi di errore e codici di stato

Codice	Descrizione
200 OK	Operazione riuscita
201 Created	Creazione avvenuta con successo
400 Bad Request	Dati non validi
401 Unauthorized	Token mancante o non valido
403 Forbidden	Accesso negato (ruolo non autorizzato)
404 Not Found	Risorsa inesistente
500 Internal Server Error	Errore interno del server

3.5 API - User

La risorsa User rappresenta qualunque utente registrato nel sistema (Customer o Restaurateur).

Le operazioni disponibili consentono la registrazione, l'autenticazione, la gestione del profilo personale e la rimozione dell'account.

Le API gestiscono ruoli diversi attraverso il campo role (CUSTOMER, RESTAURATEUR).

3.5.1 POST /api/lv/users/register

Crea un nuovo account utente.

Al termine della creazione, il sistema restituisce un campo *nextStep* che indica la pagina frontend di completamento del profilo in base al ruolo scelto.

Se *role* = CUSTOMER → il campo *nextStep* assume valore */complete-profile/customer*

Se *role* = RESTAURATEUR → il campo *nextStep* assume valore */complete-profile/restaurateur*

La richiesta va compilata con: firstName, lastName, email, password, phoneNumber, role.

La risposta conterrà: id, email, role.

Errori possibili: 400, 409.

3.5.2 POST /api/lv/users/login

Permette l'autenticazione dell'utente e restituisce un token JWT da usare per le operazioni protette.

La richiesta va compilata con: email, password.

La risposta conterrà: id, role.

Errori possibili: 401, 404

3.5.3 GET /api/lv/users/me

Restituisce le informazioni del profilo dell'utente autenticato.

Richiede un token JWT valido.

La password viene salvata nel db come hash crittografico.

Il token JWT include i campi id, email, role e scade dopo 24h.

3.5.4 PUT api/lv/users/me

Aggiorna i dati personali dell'utente autenticato

La richiesta va compilata con i dati che si desidera aggiornare.

La risposta consisterà in un messaggio di successo o fallimento.

3.5.5 DELETE /api/lv/users/me

Elimina definitivamente l'account dell'utente autenticato e i dati collegati.

La richiesta va compilata con l'id dell'utente autenticato

La risposta consisterà in un messaggio di successo (200) o fallimento.

3.6 API - Restaurant

La risorsa *Restaurant* rappresenta i locali gestiti dai ristoratori registrati.

Ogni ristoratore autenticato può creare più ristoranti, modificarne o eliminarne i dati.

Gli endpoint relativi sono protetti da autenticazione JWT e dal ruolo RESTAURATEUR

3.6.1 POST /api/lv/restaurants

Crea un nuovo ristorante associato al ristoratore autenticato.

La richiesta va compilata con le informazioni relative al ristorante.

La risposta restituisce l'id del ristorante creato o un codice di errore.

Errori possibili: 400, 409.

3.6.2 GET api/lv/restaurants

Restituisce l'elenco di tutti i ristoranti presenti nel sistema.

L'accesso è pubblico per la consultazione da parte di visitatori e clienti.

3.6.3 GET /api/lv/restaurants/:id

Restituisce i dettagli di un singolo ristorante incluso il menù associato.

Accesso pubblico.

3.6.4 GET /api/lv/restaurants/my-restaurants

Restituisce un elenco dei ristoranti associati all'account del *RESTAURATEUR* autenticato.

Errori possibili: 401, 403

3.6.5 PUT /api/lv/restaurants/:id

Aggiorna le informazioni di un ristorante appartenente al ristoratore autenticato.

Richiede il token nell'header.

Errori possibili: 403, 404.

2.3.6 DELETE /api/v1/restaurants/:id

Elimina un ristorante di proprietà del ristoratore autenticato.

L'operazione comporta la rimozione del collegamento con eventuali menù o ordini associati.

Restituisce errore se vi sono ordini non evasi.

Errori possibili: 403, 404.

3.7 API - Menu

La risorsa *Menu* rappresenta l'insieme dei piatti offerti da uno o più ristoranti gestiti da un ristoratore.

Ogni menù è associato a uno o più ristoranti e può essere condiviso o duplicato in fase di creazione di nuovi locali.

3.7.1 POST /api/lv/menu

Crea un nuovo menù e lo associa a uno o più ristoranti appartenenti al ristoratore autenticato.

Se il ristoratore crea un nuovo ristorante, il sistema può chiedere se importare un menù già esistente; in tal caso viene creato un nuovo documento menù con gli stessi *dishId* ma differente *restaurantId*.

L'accesso è limitato ad utenti autenticati come *RESTAURATEUR*.

Errori possibili: 400, 403.

3.7.2 GET /api/lv/menu/:id

Restituisce il dettaglio di un menù includendo le informazioni sui piatti associati.

Errori possibili: 404

3.7.3 PUT /api/lv/menu/:id

Aggiorna il contenuto del menu aggiungendo, modificando o rimuovendo i piatti.

L'accesso è limitato ad utenti autenticati come *RESTAURATEUR*.

Se il menù è condiviso tra più ristoranti, il ristoratore viene avvisato e può scegliere se applicare la modifica solo al menù corrente, a tutti i menù collegati o ad una porzione di essi.

3.7.4 DELETE /api/lv/menu/:id

Elimina un menù.

L'accesso è limitato ad utenti autenticati come *RESTAURATEUR*.

L'operazione è consentita solo se non sono presenti ordini attivi legati ai piatti di quel menù.

3.8 API - Dish

La risorsa *Dish* rappresenta i piatti disponibili nel sistema.

Ogni piatto può provenire dal catalogo generale importato da *meals.json* o dai piatti personalizzati creati da un ristoratore per i propri menù.

Il sistema utilizza il file `meals.json` come catalogo base di piatti globali (dataset fornito all'avvio dell'applicazione).

I piatti importati da questo file sono marcati nel database con *source = catalog* e vengono trattati come risorse di sola lettura. Quando un ristorante seleziona un piatto dal catalogo per aggiungerlo al proprio menù, il sistema non modifica il documento originale, ma crea una copia personalizzata

Gli endpoint permettono la consultazione, creazione, modifica ed eliminazione dei piatti.

Le operazioni di scrittura sono riservate al ruolo *RESTAURATEUR*.

3.8.1 GET /api/lv/dishes

Restituisce la lista di tutti i piatti disponibili, sia del catalogo generale sia quelli aggiunti dai ristoranti comprensiva di riferimenti ai ristoranti che offrono il piatto.

Supporta parametri di filtro per nome, categoria, prezzo e ingredienti.

3.8.2 GET /api/lv/dishes/:id

Restituisce i dettagli completi di un singolo piatto comprensive di ristoranti in cui è possibile acquistarlo.

Errori possibili: 404.

3.8.3 POST /api/lv/dishes

Permette al ristorante autenticato di creare un nuovo piatto personalizzato da aggiungere al proprio menù.

3.8.4 PUT /api/lv/dishes/:id

Aggiorna i dettagli di un piatto esistente collegato ad un menù appartenente al ristorante autenticato.

Errori possibili: 403, 404

3.8.5 DELETE /api/lv/dishes/:id

Elimina un piatto personalizzato creato dal ristorante autenticato.

I piatti importati da `meals.json` non possono essere rimossi, ma solo duplicati o esclusi dal menù.

Se un piatto è collegato ad ordini attivi restituisce errore 409.

Errori possibili: 403, 409.

3.9 API - Order

La risorsa Order gestisce l'intero ciclo di vita di un ordine effettuato da un cliente presso un ristorante.

Ogni ordine è associato a un solo cliente e a un solo ristorante, e può contenere uno o più piatti provenienti dal relativo menù.

Gli stati di un ordine sono:

ORDINATO → *IN_PREPARAZIONE* → *CONSEGNATO*

Il cliente può annullare un ordine solo se lo stato è "ORDINATO".

Tutte le API richiedono autenticazione JWT.

3.9.1 POST /api/lv/orders

Crea un nuovo ordine per l'utente autenticato con ruolo *CUSTOMER*.

Setta lo stato dell'ordine come *ORDINATO*

Il campo `totalPrice` viene calcolato automaticamente sommando i prezzi dei piatti moltiplicati per la quantità.

L'ordine viene collegato sia al *customerId* che al *restaurantId*.

3.9.1 GET /api/lv/orders

Restituisce tutti gli ordini associati all'utente autenticato.

Il comportamento varia a seconda del ruolo: il customer visualizza lo storico dei propri ordini, al ristorante viene chiesto di filtrare per singolo ristorante e poi viene mostrato lo storico relativo al singolo ristorante comprensivo di stato degli ordini.

3.9.2 GET /api/lv/orders/:id

Restituisce i dettagli completi di un singolo ordine, inclusi i piatti ordinati e le date di creazione/aggiornamento.

3.9.3 PATCH /api/lv/orders/:id/status

Aggiorna lo stato di un ordine.

Accessibile solo ai ristoratori per gli ordini associati ai propri ristoranti.

3.9.4 PATCH /api/v1/orders/:id/cancel

Permette al cliente di annullare un ordine solo se lo stato è ancora "ORDINATO".

Errori possibili: 403, 404.

4. Sicurezza e autenticazione

Il sistema *Leaveat* implementa un modello di autenticazione basato su *JSON Web Token* (JWT).

Ogni utente, dopo il login, riceve un token firmato che deve essere incluso nell'header di ogni richiesta protetta: *Authorization: Bearer <token>*.

Il token contiene: *userId*, *email* e *role* (CUSTOMER o RESTAURATEUR) ed ha una durata predefinita di 24 ore.

Alla scadenza, l'utente deve effettuare un nuovo login.

4.1 Middleware di autenticazione

Un middleware Express intercetta ogni richiesta alle rotte protette e:

1. verifica la presenza del token;
2. ne valida la firma;
3. estrae i dati dell'utente autenticato e li aggiunge a *req.user*.

Se il token è assente o invalido → risposta 401 Unauthorized

4.3 Autorizzazione per ruoli

Un secondo middleware controlla i permessi in base al ruolo:

CUSTOMER → accesso a ordini personali, consultazione ristoranti e menu.

RESTAURATEUR → accesso a gestione ristoranti, menu e ordini ricevuti.

In caso di ruolo non autorizzato → risposta 403 Forbidden

5. Gestione errori e popolamento iniziale

5.1 Gestione degli errori

Tutte le API restituiscono risposte strutturate con i campi:

```
{ success: false, error: "ValidationError", message: "Campo mancante" }.
```

Gli errori vengono gestiti da un middleware globale che intercetta eccezioni non catturate e restituisce un codice HTTP appropriato.

5.2 Popolamento iniziale (setup)

Alla prima esecuzione del backend, un modulo di setup carica nel database i dati di base da *meals.json*.

I piatti vengono salvati nella collezione *dishes* con *source: "catalog"*.

Gli ingredienti e le misure vengono convertiti in array.

I campi *idMeal*, *strMeal*, *strCategory* e *strArea* vengono mappati rispettivamente su *externalId*, *name*, *category* e *area*.

In caso di errori durante l'import, il sistema registra i log nel file *setup.log*